Kubernetes Documentation: Karpenter, Dynamic Provisioning & Autoscaling

Overview

This documentation covers a Kubernetes implementation on AWS EKS in **ap-south-1**, focusing on:

- **Karpenter** for node provisioning
- **Dynamic node provisioning** driven by pod scheduling needs
- **Cluster autoscaling** using **HPA (pods)** + **Karpenter (nodes)**

Tooling & assumptions

- Cluster created with `eksctl`
- Cluster operated with `kubectl`
- Karpenter installed via `helm`
- AWS credentials configured locally (e.g., via `aws configure` or SSO)

---

1) Architecture Overview

Objective

Describe how EKS, Karpenter, and HPA interact end-to-end to deliver elastic capacity (pods + nodes) without manual node group sizing.

System architecture

EKS control plane (managed by AWS)

- **API Server**: entry point for all cluster operations (`kubectl`, controllers, scheduler).
- **etcd**: stores cluster state (deployments, pods, HPAs, nodes, RBAC, etc.).
- **Controller Manager**: runs core controllers (replica set controller, node controller, endpoints controller, etc.).
- **Scheduler**: assigns pods to nodes based on resources and constraints.

Operational implication: you don't manage control-plane instances directly; availability, patching, and scaling are handled by AWS.

Worker nodes (data plane)

- EC2 instances that run:
  - `kubelet` (node agent)
  - container runtime (e.g., containerd)
  - `kube-proxy` (service networking)
  - VPC CNI components (pod IP allocation/routing)

Karpenter controller

- Runs as pods in the cluster (namespace: `karpenter`).
- Watches for scheduling failures / unschedulable pods.
- Calls AWS EC2 APIs to provision instances that satisfy pod scheduling requirements.

HPA (Horizontal Pod Autoscaler)

- Uses CPU (and/or memory/custom metrics) to adjust **replica count**.
- Requires Metrics Server (or a metrics pipeline) to provide resource utilization.

AWS EC2 interaction

Karpenter typically interacts with AWS through:

- **EC2 RunInstances / Fleet-like logic**: to launch capacity.

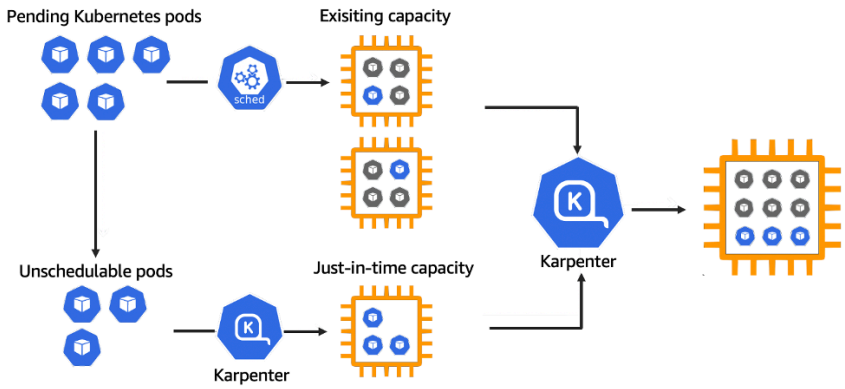
- **Tags + selectors**: to choose subnets and security groups.
- **IAM (via IRSA/OIDC)**: to authorize these API calls without static credentials.

End-to-end flow

Request → Pod creation → Scheduling → Pending → Karpenter → Node creation → Pod scheduled

Detailed sequence:

1. A user applies a workload (e.g., `kubectl create deployment nginx --image=nginx`).
2. The Deployment creates a ReplicaSet which creates Pod objects.
3. The Scheduler attempts to bind each Pod to a node.
4. If no node matches (insufficient CPU/memory, constraints, or no nodes exist), the Pod stays **Pending** and events indicate it is unschedulable.
5. Karpenter observes the unschedulable state and computes a node shape that can satisfy pod requests and constraints (and your Provisioner requirements).
6. Karpenter provisions an EC2 instance in the selected subnets/security groups.
7. The new node boots, joins the cluster, and becomes **Ready**.
8. The Scheduler retries and binds the Pending pods to the new node.



---

2) AWS Component Breakdown

VPC

Why required: EKS worker nodes and pods need networking (IP ranges, routing, egress).

How EKS uses it internally:

- Worker nodes are launched into VPC subnets.
- The Kubernetes VPC CNI assigns pod IPs from the VPC network.

Subnets

Why required: determine where worker nodes/pods get IPs and what routing rules apply.

How EKS uses them internally:

- Subnets are associated with the cluster and used for node placement.
- Karpenter uses your `subnetSelector` to pick eligible subnets for new nodes.

Security Groups

Why required: control inbound/outbound traffic to nodes and control-plane-to-node communication.

How EKS uses them internally:

- Nodes attach security groups; required ports must allow kubelet/API server traffic.
- Karpenter uses `securityGroupSelector` to attach the correct SGs to provisioned nodes.

IAM Roles (conceptual)

Why required: authorize AWS API actions.

- EKS nodes need permissions for ECR pulls, CNI networking, logging, etc.
- Karpenter needs permissions to create/terminate EC2 instances and read related resources.

How EKS uses them internally:

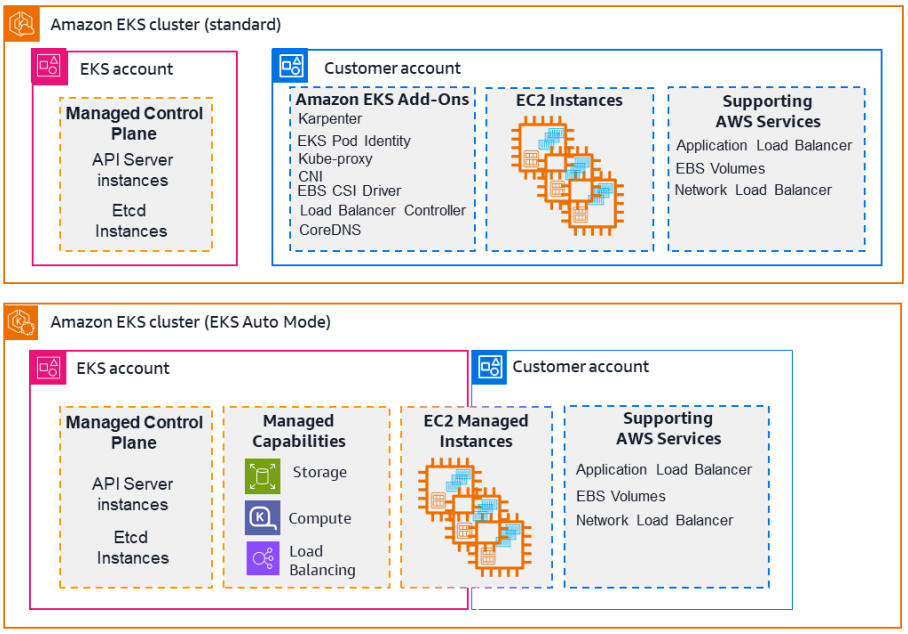
- Node IAM role is assumed by the EC2 instance profile.
- IRSA binds a Kubernetes service account to an IAM role using OIDC.

EC2 instances

Why required: they are the compute capacity that runs Kubernetes workloads.

How EKS uses them internally:

- Instances run kubelet and become Kubernetes Nodes.
- Pod scheduling consumes CPU/memory available on these nodes.



---

3) Security Considerations

Why IAM roles are required

Karpenter must call AWS APIs to launch/terminate instances; nodes and add-ons also need AWS permissions.

Risks of using root user

- Root credentials bypass least-privilege controls.
- High blast radius if compromised.
- Difficult to audit responsibly compared to role-based access.

Role-based access (IRSA concept)

- OIDC enables Kubernetes service accounts to assume IAM roles.
- Each controller/workload can receive only the permissions it needs.
- Eliminates static AWS keys inside pods.

Network security basics (security groups)

- Restrict node inbound traffic to required ports.
- Control egress to required AWS endpoints/services.
- Keep management access (SSH) closed unless explicitly required.

---

4) Cost Analysis

EKS control plane cost

- EKS charges for the managed control plane per cluster (pricing varies by region).
- This cost is fixed regardless of worker node count.

EC2 cost estimation (practical approach)

Costs depend on:

- Instance type(s) selected (e.g., `t3.small`, `t3.medium`)
- Number of nodes and their runtime hours
- Storage (EBS) and data transfer

Practical method:

1. Estimate average nodes during business hours vs off-hours.
2. Multiply by hourly EC2 rate for selected types.
3. Add storage and any NAT/data egress where applicable.

How Karpenter reduces cost

- Reduces idle capacity by provisioning only when pods require it.
- Avoids “one-size-fits-all” node groups by selecting the smallest viable instances within constraints.
- Enables better bin-packing (fewer wasted resources).

Best practices to minimize cost

- Keep Provisioner instance type list curated (avoid expensive types unless needed).
- Set `limits.resources` to prevent runaway provisioning.
- Right-size pod requests/limits (over-requesting CPU causes unnecessary nodes).
- Use separate provisioning constraints for different workload tiers (prod vs dev).

---

5) Monitoring and Logging

How to monitor the cluster

Start with `kubectl`:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl get nodes
kubectl describe node <node-name>
kubectl logs -n karpenter deployment/karpenter
```

Events-driven troubleshooting

```
kubectl get events --sort-by=.lastTimestamp
```

Prometheus / Grafana (conceptual)

- **Prometheus**: collects time-series metrics from cluster components and workloads.
- **Grafana**: visualizes dashboards (node utilization, HPA behavior, scheduling latency, controller errors).

---

6) Failure Scenarios

Node crash

What happens

- Pods on the node are disrupted.
- Node controller marks node NotReady; pods are rescheduled if allowed.

How the system recovers

- Scheduler reschedules pods to other nodes if capacity exists.
- If capacity does not exist, pods remain Pending and Karpenter can provision replacement capacity.

Karpenter failure

What happens

- Pending pods remain Pending because no controller provisions new nodes.

How the system recovers

- Restore Karpenter deployment; once healthy, it reconciles pending pods.

Pods not scaling

What happens

- Replica count stays flat even under load.

Common causes

- Metrics Server not available
- CPU requests not set appropriately (HPA uses requests)
- HPA misconfiguration

API throttling

What happens

- Node provisioning can slow down; Karpenter may retry.

How the system recovers

- Retries succeed when throttling subsides.
- Reduce bursty provisioning; review EC2 API usage and constraints.

---

7) Internal Working (Deep Dive)

How the Kubernetes scheduler works (practical view)

At a high level, the scheduler performs:

1. **Queueing**: new Pods without a `nodeName` enter the scheduling queue.
2. **Filtering**: eliminates nodes that cannot run the pod.
    - Insufficient resources (CPU/memory)
    - Taints/tolerations
    - Node selectors / node affinity
    - Topology spread constraints
    - Volume binding constraints (if applicable)

3. **Scoring**: ranks remaining nodes to pick the best fit (resource balance, spreading, etc.).
4. **Binding**: writes the binding decision (pod → node) back to the API server.

What happens when pods are unschedulable

When no nodes pass filtering, the scheduler:

- Leaves the pod in **Pending**.
- Records an event such as:
  - `0/1 nodes are available: insufficient cpu.`
  - `node(s) didn't match node selector.`
- Re-queues the pod for retry (scheduler periodically re-attempts).

Runbook commands to inspect:

```
kubectl get pods
kubectl describe pod <pod-name>
```

How Karpenter detects pending pods

Karpenter watches the Kubernetes API for pods that are:

- Pending
- Unschedulable (based on scheduler events/conditions)

Karpenter then performs capacity planning against:

- Pod resource requests (CPU/memory)
- Pod constraints (selectors/affinities/tolerations)
- Provisioner requirements (allowed instance types, limits)

How instance type selection happens

Instance selection is driven by constraints and availability:

- Your Provisioner restricts candidates:

    - `requirements` (e.g., only `t3.small`, `t3.medium`)
    - optional additional constraints in real setups (arch, OS, capacity type)
- Karpenter chooses a viable instance type that can fit:
    - the pod's requested resources
    - daemonset overhead on the node
    - bin-packing considerations (packing multiple pods efficiently)

If no candidates exist (too restrictive), pods remain Pending.

How nodes are joined to the cluster

Once an EC2 instance launches:

1. Bootstrap/user-data config installs/starts required node components.
2. The node registers with the API server (kubelet registration).
3. The node becomes `Ready` once:
    - kubelet is healthy
    - networking is configured
    - required system pods/daemonsets are running

Validation commands:

```
kubectl get nodes
kubectl describe node <node-name>
```

---

8) Comparison: Karpenter vs Cluster Autoscaler

Comparison table

Dimension	Karpenter	Cluster Autoscaler
Speed	Typically faster scale-out due to direct provisioning and flexible selection	Depends on node group scaling and available group configuration
Flexibility	High — can select instance types dynamically (within constraints)	Medium — limited to node group instance types and sizes
Cost optimization	Strong — better bin-packing and right-sizing; supports advanced strategies	Good — but often requires multiple tuned node groups
Node group dependency	Low — does not require many pre-created node groups for different shapes	High — depends on node groups; best results often need multiple node groups
Real-world usage	Elastic, workload-driven compute (burst/spiky workloads, multi-tenant clusters)	Stable scaling based on configured groups (common in traditional EKS setups)

Practical guidance

- If you want minimal node group management and workload-driven node shapes, Karpenter is a strong fit.
- If your organization standardizes on fixed node groups and you need mature, well-known behavior, Cluster Autoscaler remains viable.

---

9) Karpenter

Definition

Karpenter is a Kubernetes node lifecycle controller that provisions and deprovisions nodes in response to unschedulable pods.

Overview

- Watches for pods that cannot be scheduled due to insufficient capacity/constraints

- Provisions EC2 instances that match pod requirements (instance type, capacity, constraints)
- Consolidates/removes nodes when they're underutilized (depending on configuration)

Rationale

- Faster scale-out than traditional autoscalers in many scenarios
- Better cost/performance optimization by choosing right-sized instances
- Reduces operational overhead managing node groups

Placement & Dependencies

- Runs as a controller inside the EKS cluster (typically in a dedicated namespace, e.g., `karpenter`)
- Interfaces with AWS via IAM/OIDC (IRSA) and EC2 APIs

Implementation Procedure (Step-by-step)

Step 1 — Create EKS cluster

Create an EKS cluster using `eksctl` with a managed node group.

```
eksctl create cluster \
  --name karpenter-cluster \
  --region ap-south-1 \
  --nodes 1 \
  --node-type t3.small \
  --managed
```

```
2026-04-12 13:32:13 [!]  Auto Mode will be enabled by default in an upcoming release of eksctl. This means managed node groups and managed networking add-ons will no longer be created
 by default. To maintain current behavior, explicitly set `autoModeConfig.enabled: false` in your cluster configuration. Learn more: https://eksctl.io/usage/auto-mode/
2026-04-12 13:32:13 [ℹ]  using Kubernetes version 1.34
2026-04-12 13:32:13 [ℹ]  creating EKS cluster "karpenter-cluster" in "ap-south-1" region with managed nodes
2026-04-12 13:32:13 [ℹ]  will create 2 separate CloudFormation stacks for cluster itself and the initial managed nodegroup
2026-04-12 13:32:13 [ℹ]  if you encounter any issues, check CloudFormation console or try 'eksctl utils describe-stacks --region=ap-south-1 --cluster=karpenter-cluster'
2026-04-12 13:32:13 [ℹ]  Kubernetes API endpoint access will use default of {publicAccess=true, privateAccess=false} for cluster "karpenter-cluster" in "ap-south-1"
2026-04-12 13:32:13 [ℹ]  CloudWatch logging will not be enabled for cluster "karpenter-cluster" in "ap-south-1"
2026-04-12 13:32:13 [ℹ]  you can enable it with 'eksctl utils update-cluster-logging --enable-types={SPECIFY-YOUR-LOG-TYPES-HERE (e.g. all)} --region=ap-south-1 --cluster=karpenter-cl
uster'
2026-04-12 13:32:13 [ℹ]  default addons kube-proxy, coredns, metrics-server, vpc-cni were not specified, will install them as EKS addons
2026-04-12 13:32:13 [ℹ]
2 sequential tasks: { create cluster control plane "karpenter-cluster",
    2 sequential sub-tasks: {
        2 sequential sub-tasks: {
            1 task: { create addons },
            wait for control plane to become ready,
        },
        create managed nodegroup "ng-95bce64b",
    }
}
2026-04-12 13:32:13 [ℹ]  building cluster stack "eksctl-karpenter-cluster-cluster"
2026-04-12 13:32:13 [ℹ]  deploying stack "eksctl-karpenter-cluster-cluster"
2026-04-12 13:32:43 [ℹ]  waiting for CloudFormation stack "eksctl-karpenter-cluster-cluster"
```

```
nded way to provide IAM permissions for `vpc-cni` addon is via pod identity associations; after addon creation is completed, add all recommended policies to the config file, under `ad
don.PodIdentityAssociations`, and run `eksctl update addon`
2026-04-12 13:41:16 [ℹ]  creating addon: vpc-cni
2026-04-12 13:41:16 [ℹ]  successfully created addon: vpc-cni
2026-04-12 13:43:16 [ℹ]  building managed nodegroup stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:43:17 [ℹ]  deploying stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:43:17 [ℹ]  waiting for CloudFormation stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:43:47 [ℹ]  waiting for CloudFormation stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:44:35 [ℹ]  waiting for CloudFormation stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:46:35 [ℹ]  waiting for CloudFormation stack "eksctl-karpenter-cluster-nodegroup-ng-95bce64b"
2026-04-12 13:46:35 [ℹ]  waiting for the control plane to become ready
2026-04-12 13:46:35 [✓]  saved kubeconfig as "/home/cloudshell-user/.kube/config"
2026-04-12 13:46:35 [ℹ]  no tasks
2026-04-12 13:46:35 [✓]  all EKS cluster resources for "karpenter-cluster" have been created
2026-04-12 13:46:36 [ℹ]  nodegroup "ng-95bce64b" has 1 node(s)
2026-04-12 13:46:36 [ℹ]  node "ip-192-168-58-140.ap-south-1.compute.internal" is ready
2026-04-12 13:46:36 [ℹ]  waiting for at least 1 node(s) to become ready in "ng-95bce64b"
2026-04-12 13:46:36 [ℹ]  nodegroup "ng-95bce64b" has 1 node(s)
2026-04-12 13:46:36 [ℹ]  node "ip-192-168-58-140.ap-south-1.compute.internal" is ready
2026-04-12 13:46:36 [✓]  created 1 managed nodegroup(s) in cluster "karpenter-cluster"
2026-04-12 13:46:36 [ℹ]  creating addon: metrics-server
2026-04-12 13:46:37 [ℹ]  successfully created addon: metrics-server
2026-04-12 13:46:38 [ℹ]  kubectl command should work with "/home/cloudshell-user/.kube/config", try 'kubectl get nodes'
2026-04-12 13:46:38 [✓]  EKS cluster "karpenter-cluster" in "ap-south-1" region is ready
~ $
```

Parameter breakdown

- `--name karpenter-cluster`: Sets the EKS cluster name.
- `--region ap-south-1`: Creates the cluster in AWS region **ap-south-1 (Mumbai)**.
- `--nodes 1`: Creates an initial node group size of **1** node.
- `--node-type t3.small`: Instance type for the initial node(s).
- `--managed`: Uses an **EKS Managed Node Group** (AWS manages the underlying Auto Scaling Group lifecycle).

Step 2 — Configure kubectl

Update local kubeconfig to point `kubectl` at the cluster.

```
aws eks update-kubeconfig --name karpenter-cluster --region a
p-south-1
```

Validate cluster connectivity and node readiness.

```
kubectl get nodes
```

```
~ $ kubectl get nodes
NAME                                          STATUS   ROLES    AGE     VERSION
ip-192-168-58-140.ap-south-1.compute.internal   Ready    <none>   3m59s   v1.34.4-eks-f69f56f
~ $
```

Expected: at least one node in **Ready** state.

Step 3 — Enable OIDC (IRSA prerequisite)

Associate an IAM OIDC provider with the cluster.

```
eksctl utils associate-iam-oidc-provider \
  --region ap-south-1 \
  --cluster karpenter-cluster \
  --approve
```

```
~ $ eksctl utils associate-iam-oidc-provider \
>   --region ap-south-1 \
>   --cluster karpenter-cluster \
>   --approve
2026-04-12 14:16:23 [ℹ]  will create IAM Open ID Connect provider for cluster "karpenter-cluster" in "ap-south-1"
2026-04-12 14:16:23 [✓]  created IAM Open ID Connect provider for cluster "karpenter-cluster" in "ap-south-1"
~ $
```

Why OIDC is required for Karpenter

- Karpenter needs AWS API access (EC2, IAM, etc.) to launch and manage instances.
- Best practice on EKS is to use **IRSA (IAM Roles for Service Accounts)**.
- IRSA requires the EKS cluster to have an **OIDC identity provider** associated so Kubernetes service accounts can assume IAM roles securely.

Step 4 — Install Helm

Install Helm 3 using the official install script.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
~ $ helm version
-bash: helm: command not found
~ $ curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 11929 100 11929    0     0 669980     0  --:--:-- --:--:-- --:--:-- 701705
Downloading https://get.helm.sh/helm-v3.20.2-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
~ $ helm version
version.BuildInfo{Version:"v3.20.2", GitCommit:"8fb76d6ab555577e98e23b7500009537a471feee", GitTreeState:"clean", GoVersion:"go1.25.9"}
~ $
```

Step 5 — Add Karpenter Helm repo

Add the Karpenter chart repository and update indices.

```
helm repo add karpenter https://charts.karpenter.sh/
helm repo update
```

```
~ $ helm repo add karpenter https://charts.karpenter.sh
"karpenter" has been added to your repositories
~ $ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "karpenter" chart repository
Update Complete. ⎈Happy Helming!⎈
~ $
```

Step 6 — Create namespace

Create a dedicated namespace for Karpenter.

```
kubectl create namespace karpenter
```

Step 7 — Install Karpenter

Install Karpenter into the `karpenter` namespace.

```
helm install karpenter karpenter/karpenter --namespace karpen
ter
```

```
~ $ helm install karpenter karpenter/karpenter \
>   --namespace karpenter \
>   --create-namespace \
>   --set serviceAccount.name=karpenter \
>   --set settings.clusterName=$CLUSTER_NAME \
>   --set settings.clusterEndpoint=$CLUSTER_ENDPOINT \
>   --set settings.aws.defaultInstanceProfile=KarpenterNodeInstanceProfile
NAME: karpenter
LAST DEPLOYED: Sun Apr 12 14:25:31 2026
NAMESPACE: karpenter
STATUS: deployed
REVISION: 1
TEST SUITE: None
~ $
```

Validate pods are running.

```
kubectl get pods -n karpenter
```

```
~ $ kubectl get pods -n karpenter
NAME                          READY   STATUS    RESTARTS   AGE
karpenter-7766866cb7-n9gkh    1/1     Running   0          2m59s
karpenter-7766866cb7-qhm4m    1/1     Running   0          3m
~ $
```

Step 8 — Create Provisioner

Create a `Provisioner` resource defining how Karpenter should provision nodes.

Create `provisioner.yaml`:

```yaml
apiVersion: karpenter.sh/v1alpha5
kind: Provisioner
metadata:
  name: default
spec:
  requirements:
  - key: "node.kubernetes.io/instance-type"
    operator: In
    values: ["t3.small","t3.medium"]

  limits:
    resources:
      cpu: 1000

  provider:
    subnetSelector:
      kubernetes.io/cluster/karpenter-cluster: "*"
    securityGroupSelector:
      kubernetes.io/cluster/karpenter-cluster: "*"
```

Apply it:

```
kubectl apply -f provisioner.yaml
```

```
~ $ cat <<EOF | kubectl apply -f -
> apiVersion: karpenter.sh/v1alpha5
> kind: Provisioner
> metadata:
>   name: default
> spec:
>   requirements:
>     - key: "node.kubernetes.io/instance-type"
>       operator: In
>       values: ["t3.small","t3.medium"]
>   limits:
>     resources:
>       cpu: 1000
>   provider:
>     instanceProfile: KarpenterNodeInstanceProfile
>   ttlSecondsAfterEmpty: 30
> EOF
provisioner.karpenter.sh/default created
~ $
```

Notes on this Provisioner

- `requirements`: constrains instances to `t3.small` or `t3.medium`.
- `limits.resources.cpu`: caps total CPU provisioned by this provisioner (safety guardrail).
- `provider.subnetSelector` and `provider.securityGroupSelector`: selects cluster-tagged subnets and security groups.

Use Cases

- Burst scaling for batch jobs, CI runners, or spiky workloads
- Cost optimization by using the smallest instance types that meet requirements
- Rapid provisioning when pods become Pending due to capacity constraints

---

10) Dynamic Node Provisioning

Definition

Dynamic node provisioning is the practice of automatically creating and removing cluster nodes in response to real-time scheduling demand.

Overview

- Nodes are not pre-provisioned for peak load.
- When pods are Pending due to insufficient resources or constraints, Karpenter provisions new nodes that satisfy those constraints.
- When demand drops, nodes can be removed (depending on Karpenter configuration and workload constraints).

Rationale

- Avoids over-provisioning fixed node groups.
- Improves responsiveness to workload spikes.
- Enables flexible compute choices (instance types) per workload needs.

Placement & Dependencies

- Triggered by the Kubernetes scheduler (pods becoming **Unschedulable/Pending**)
- Implemented by Karpenter controller + AWS EC2 provisioning

Implementation Procedure (Step-by-step)

Step 9 — Deploy sample application

Deploy a simple `nginx` workload.

```
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --port 80 --type ClusterIP
```

Validate pods and service.

```
kubectl get pods
kubectl get svc
```

Step 13 (part 1) — Observe dynamic node creation

As load increases (see autoscaling section), observe:

```
kubectl get pods
kubectl get nodes
```

What to look for:

- Pods temporarily Pending due to lack of capacity
- New nodes appearing (launched by Karpenter)
- Pending pods being scheduled onto newly created nodes

Use Cases

- Event-driven workloads (flash sales, marketing spikes)
- Microservices platforms where services scale at different rates
- Multi-tenant clusters needing elastic capacity without large fixed node pools

---

11) Cluster Autoscaling (HPA + Karpenter)

Definition

Cluster autoscaling in this project means:

- **HPA** scales *pods* based on CPU utilization.
- **Karpenter** scales *nodes* to provide capacity for those pods.

Overview

- HPA increases replicas when CPU exceeds a threshold.
- If the cluster lacks resources to run new replicas, pods become Pending.
- Karpenter detects unschedulable pods and provisions nodes to fit them.

Rationale

- HPA alone cannot add nodes; it only changes pod replica counts.
- Node capacity must also scale; Karpenter provides that elasticity.

Placement & Dependencies

- HPA runs as Kubernetes control-plane functionality.
- Metrics Server provides resource metrics (CPU/memory) needed by HPA.
- Karpenter provisions nodes on AWS to satisfy scheduling requirements.

Implementation Procedure (Step-by-step)

Step 10 — Install Metrics Server

Install Metrics Server (required for HPA CPU-based scaling).

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-s
erver/releases/latest/download/components.yaml
```

Validate metrics availability (may take a short time to become ready):

```
kubectl top nodes
kubectl top pods
```

Step 11 — Apply HPA

Create an HPA for the `nginx` deployment.

```
kubectl autoscale deployment nginx \
  --cpu-percent=50 \
  --min=1 \
  --max=10
```

Check HPA status:

```
kubectl get hpa
```

Step 12 — Generate load

Run a load generator pod and continuously call nginx.

```
kubectl run -i --tty load-generator --image=busybox -- /bin/s
h
```

Inside the container:

```
while true; do wget -q -O- http://nginx/; done
```

Observe CPU/memory utilization (in another terminal):

```
kubectl top pods
kubectl top nodes
```

Step 13 — Observe autoscaling

Observe pods and nodes:

```
kubectl get pods
kubectl get nodes
```

How to interpret results

- **Pods scaling (HPA):** `nginx` replica count increases when average CPU exceeds 50%.
- **Nodes created (Karpenter):** when additional replicas can't be scheduled, Karpenter creates new nodes that match the Provisioner requirements (e.g., `t3.small`, `t3.medium`).

Use Cases

- Web workloads with variable traffic (API gateways, frontends)
- Services with predictable CPU thresholds

- Environments needing automatic “pods + capacity” scaling without manual node group tuning

---

12) Step-by-Step Execution Flow (Narrative)

1. A user deploys or updates an application (e.g., `nginx`).
2. Traffic increases; CPU utilization rises.
3. Metrics Server reports the utilization.
4. HPA evaluates the 50% target and increases replicas.
5. New pods are created, but the scheduler cannot place all of them on existing nodes.
6. Those pods stay **Pending**, and `kubectl describe pod` shows insufficient capacity.
7. Karpenter observes the unschedulable pods and computes required node capacity within Provisioner constraints.
8. Karpenter provisions an EC2 instance; the node joins and becomes **Ready**.
9. The scheduler places the Pending pods onto the new node.
10. Load stabilizes; HPA may scale replicas down when CPU drops.

---

13) Troubleshooting

1) kubectl not connecting

Error

- `Unable to connect to the server: ...`
- `You must be logged in to the server (Unauthorized)`

Cause

- Kubeconfig not updated / wrong context
- AWS credentials expired or wrong account/role

- Cluster name/region mismatch

Solution

1. Regenerate kubeconfig:

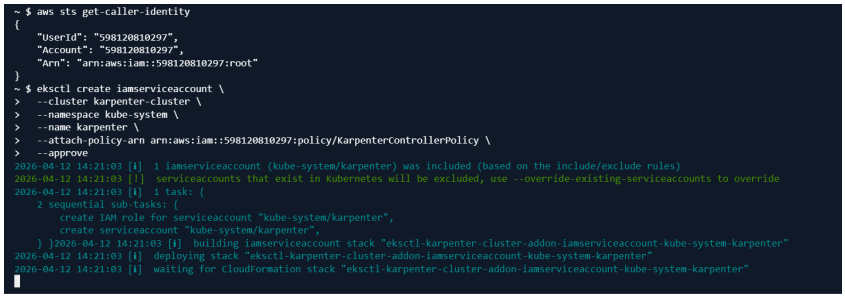
```
aws eks update-kubeconfig --name karpenter-cluster --regio
n ap-south-1
```

2. Confirm context:

```
kubectl config current-context
kubectl config get-contexts
```

3. Validate AWS identity:

```
aws sts get-caller-identity
```



2) OIDC not enabled

Error

- IRSA role assumptions failing (pods cannot assume IAM role)
- Karpenter controller logs show auth/STS errors

Cause

- OIDC provider not associated with the cluster

Solution

```
eksctl utils associate-iam-oidc-provider \
  --region ap-south-1 \
  --cluster karpenter-cluster \
  --approve
```

```
~ $ eksctl utils associate-iam-oidc-provider \
>   --region ap-south-1 \
>   --cluster karpenter-cluster \
>   --approve
2026-04-12 14:16:23 [ℹ]  will create IAM Open ID Connect provider for cluster "karpenter-cluster" in "ap-south-1"
2026-04-12 14:16:23 [✓]  created IAM Open ID Connect provider for cluster "karpenter-cluster" in "ap-south-1"
~ $
```

3) Karpenter not creating nodes

Error

- Pods remain Pending
- No new nodes appear

Cause

- Provisioner misconfiguration (subnet/security group selectors don't match)
- Insufficient IAM permissions for Karpenter
- Instance types constrained too tightly (no capacity)

Solution

1. Verify Provisioner exists:

   ```
   kubectl get provisioners
   kubectl describe provisioner default
   ```

2. Check Karpenter logs:

   ```
   kubectl logs -n karpenter deployment/karpenter
   ```

3. Loosen constraints (temporarily) to validate behavior (e.g., add more instance types).

4) Pods stuck in Pending

Error

- `0/… nodes are available: insufficient cpu/memory`
- `node(s) didn't match node selector/affinity`

Cause

- Cluster lacks compute
- Pod scheduling constraints too strict

Solution

1. Describe a pending pod:

```
kubectl describe pod <pod-name>
```

2. Ensure Provisioner allows instance types that can satisfy requests.
3. Verify Karpenter is running:

```
kubectl get pods -n karpenter
```

5) Metrics Server not working

Error

- `kubectl top` shows `Metrics API not available`
- HPA remains `Unknown` / not scaling

Cause

- Metrics Server pods not running or not ready
- Cluster networking/security restrictions

Solution

1. Check Metrics Server resources:

```
kubectl get pods -n kube-system | grep metrics
```

2. Re-apply manifest:

```
kubectl apply -f https://github.com/kubernetes-sigs/metric
s-server/releases/latest/download/components.yaml
```

3. Check logs:

```
kubectl logs -n kube-system deployment/metrics-server
```

---

14) Best Practices

Cost optimization tips

- Constrain instance types to a curated list (as done in the Provisioner) to control cost.
- Use right-sized instances and avoid over-provisioning baseline capacity.
- Set sensible `limits.resources` to prevent runaway provisioning.
- Prefer scaling nodes dynamically vs. maintaining large, always-on node pools.

Why EKS is preferred over a manual EC2 Kubernetes cluster

- Managed control plane reduces operational burden (upgrades, HA, etcd management).
- Native integrations: IAM, VPC CNI, CloudWatch, managed node groups.
- Better security posture via AWS-managed primitives and well-trodden patterns like IRSA.

Importance of IAM roles (conceptual)

- Kubernetes workloads frequently need access to AWS APIs (S3, EC2, ECR, CloudWatch).
- Using IAM roles with IRSA ties permissions to a service account rather than long-lived static credentials.
- Least privilege is practical: grant only what the controller/workload needs.

---

15) Common Mistakes

- **Forgetting OIDC**: IRSA fails; AWS API calls fail.
- **Wrong/overly restrictive instance types**: no viable capacity; pods remain Pending.
- **Metrics Server not working**: `kubectl top` fails; HPA stays `Unknown`.
- **Misconfigured Provisioner**: subnet/SG selectors don't match; provisioning fails.

---

16) Future Enhancements

- **GPU workloads**: add GPU instance constraints; isolate via taints/selectors.
- **Spot instances with Karpenter**: use interruption-tolerant workloads; add capacity-type constraints.
- **Multi-AZ scaling**: ensure subnets across AZs; use topology spread constraints.
- **CI/CD integration**: automate manifests, add rollout/rollback strategies.

---

17) Interview Questions & Answers

1. **What is Karpenter?**

   A Kubernetes node provisioning controller that creates/removes nodes based on pod scheduling demand.

2. **Difference between HPA and Cluster Autoscaler?**

HPA scales pods; Cluster Autoscaler scales node groups; Karpenter provisions nodes directly.

3. **Why EKS instead of EC2-managed Kubernetes?**

   EKS provides a managed control plane and tighter AWS integrations, reducing operational overhead.

4. **What happens when pods are Pending?**

   They are not scheduled to a node; they remain Pending until capacity/constraints allow scheduling.

5. **Why is OIDC required in this setup?**

   To enable IRSA so Karpenter can assume an IAM role without static credentials.

6. **How do you verify Karpenter is provisioning nodes?**

   Check `kubectl get nodes`, Pending pods, and Karpenter controller logs.

7. **Why might Karpenter not create nodes?**

   Constraints too strict, missing permissions, or subnet/SG selectors not matching.

8. **What is IRSA?**

   IAM Roles for Service Accounts — maps a Kubernetes service account to an IAM role using OIDC.

9. **What does Metrics Server do here?**

   Provides CPU metrics used by HPA.

10. **What's the fastest way to understand a scheduling failure?**

    `kubectl describe pod <pod-name>` and review events.

11. **What does "unschedulable" mean?**

    No existing node satisfies resource + constraint requirements.

12. **How do nodes join the cluster after provisioning?**

    They bootstrap kubelet, register with the API server, and become Ready.

13. **Can HPA create new nodes?**

No — it only scales pod replicas.

14. **What's a common reason HPA doesn't scale?**

    Metrics not available or incorrect CPU requests.

15. **What should you monitor during a scale event?**

    Pending pods, node count, HPA events/replicas, and Karpenter logs.

Appendix: Quick command index

- Cluster create: `eksctl create cluster ...`
- Kubeconfig: `aws eks update-kubeconfig ...`
- Nodes: `kubectl get nodes`
- OIDC associate: `eksctl utils associate-iam-oidc-provider ...`
- Helm install: `curl ... | bash`
- Repo add/update: `helm repo add ...` / `helm repo update`
- Namespace: `kubectl create namespace karpenter`
- Karpenter install: `helm install karpenter ...`
- Provisioner apply: `kubectl apply -f provisioner.yaml`
- Sample app: `kubectl create deployment nginx ...` / `kubectl expose ...`
- Metrics Server: `kubectl apply -f ...components.yaml`
- HPA: `kubectl autoscale deployment nginx ...`
- Load: `kubectl run -i --tty load-generator ...`